

Project Proposal

Houston Flood-Aware Route Planner

Team: Sentinel Systems

Team Members: Youmbo Fatima Ane, Kailen Brown, Shawn Anthony Webb, Malik Williams

1. Project Title

Houston Flood-Aware Route Planner

2. Problem Statement

Anyone who has lived in Houston for a few years has a flooding story. Between the flat terrain, all the concrete, and rainstorms that can dump several inches in an hour, our roads flood fast and often without much warning. Hurricane Harvey is the obvious example, but honestly it doesn't take a hurricane, even a bad afternoon thunderstorm can turn an underpass into a death trap. The problem is that the apps most people actually use to get around, Google Maps, Waze, Apple Maps, have no idea any of this is happening. They'll happily route someone straight through a low-water crossing that's been flooded for hours because all they care about is the fastest path, not whether that path is safe. That gap matters because it leads to real consequences: stalled cars, expensive repairs, and every year people who drown in their own vehicles trying to get home. We think this is worth solving because the data to prevent a lot of this already exists, it's just not built into the tools people are actually navigating with.

3. Proposed Solution

Our plan is to build a route planner, basically a web app for now, that's built specifically around Houston and actually factors flooding into the routes it gives you. Instead of writing our own mapping engine from scratch, we'd lean on an existing routing API (something like Google Maps Directions, Mapbox, or the open-source OSRM) to generate the possible routes, and then layer flood data on top of that. For the flood side, we're looking at pulling from sources like the Harris County Flood Warning System, National Weather Service alerts, and TxDOT road conditions to flag segments that are currently flooded or just historically bad, like the underpasses that flood every single time it rains hard. From there, a user types in where they're starting and where they're headed, and instead of just getting the fastest route, they get routes ranked by both time and a flood-risk score, with the risky parts highlighted on the map so they know to steer clear. That's really the core idea: keep the routing part familiar, but make flooding a first-class factor instead of something the app completely ignores.

4. Project Objectives

- Hook into a mapping/routing API (Google Maps, Mapbox, or OpenStreetMap/OSRM) so the app can actually generate candidate routes between two points within the Houston area.
- Pull in real-time and historical flood data (Harris County Flood Warning System, NWS alerts, high-water reports) so we can identify which road segments are currently flooded or high-risk.
- Build a flood-risk scoring system that ranks or filters the candidate routes based on flood exposure, not just time and distance.
- Put together an interactive, map-based interface where users can see their route options, spot flood-risk segments at a glance, and switch to a different route if they want to.
- Test the system against real past flood events (past heavy-rainfall days in Houston) to see how accurate and actually useful the flood-risk routing turns out to be.

5. Expected Outcome

By the end of the project, we want to have a working prototype, not just mockups, that someone could actually open, punch in a start and end point somewhere in the Houston area, and get back route options with a real flood-risk rating attached to each one. It should have an interactive map, a backend that's actually pulling and combining live routing data with flood/weather data, and some kind of working scoring model behind the scenes, even if it's a fairly simple one at first. Alongside the app itself, we'll put together the documentation this class expects (requirements, design notes, testing results) and a demo we can walk through in our final presentation, ideally using a real past flood event or two to show the system actually catches the roads that were dangerous that day.

6. Preliminary Technologies

- **Languages:** Python (backend/data processing), JavaScript/TypeScript (frontend), SQL (database queries).
- **Frontend Framework:** React (with a mapping library such as Leaflet.js or Mapbox GL JS).
- **Backend Framework:** Flask or Django (Python) or Node.js/Express, to serve as the API layer between the frontend, routing engine, and flood-data sources.
- **Mapping/Routing APIs:** Google Maps Directions API, Mapbox, or open-source OSRM/OpenStreetMap routing engine.
- **Flood/Weather Data Sources:** Harris County Flood Warning System (harriscountyfws.org), National Weather Service (NWS) API, TxDOT road condition data, and/or NOAA rainfall data.
- **Database:** PostgreSQL with PostGIS extension (for storing and querying geospatial flood and road-segment data), or MongoDB as an alternative.

- **Other Tools/Platforms:** Git/GitHub for version control, and a cloud platform (e.g., AWS, Render, or Heroku) for deployment/hosting.

7. Initial Project Plan

Roughly, we're thinking about the project in five stages. First, actually researching what flood data and routing APIs are out there and figuring out what's realistic to work with. Second, designing the database and backend, since we'll need somewhere to store flood-risk info tied to road segments in a way we can query quickly. Third, building out the actual flood-risk scoring piece that takes routing results and flood data and turns them into something usable. Fourth, the frontend, the actual map and interface people will interact with. And fifth, putting it all together, testing it against real flood events from the past, and getting our documentation and presentation ready. As for splitting the work, we're planning to divide things up based on each person's strengths, roughly backend/data integration, the flood-risk algorithm, frontend/UI, and testing/documentation, but everyone's staying involved in planning and integration so no one is siloed off from the rest of the system.